Design note: from raw listings to a monitored release

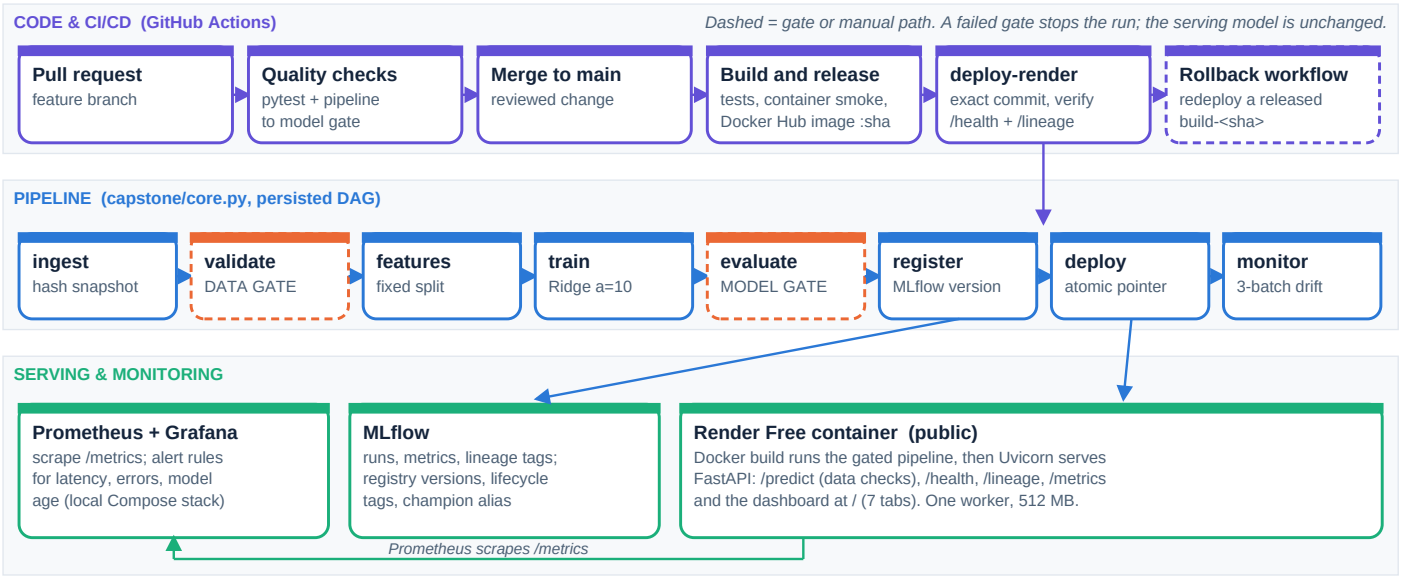
Project 4 · Capstone: End-to-End MLflow Pipeline · Swagata Bhowmik (B053) and Sumit Kothari (B062) · Dr. Santosh Kumar

01 / Objective, success metrics and non-functional requirements

A real-estate analytics start-up needs asking-price estimates (INR lakhs) for new listings, retrained and released automatically, with a record of which data, code and configuration produced each model, and a release process that refuses bad data or bad models while the previous model keeps serving.

Requirement	Target (config.yaml)	Justification and plain-language meaning for the founders
Accuracy: MAE	≤ 7.0 lakhs (validation)	"On average our estimate is within about INR 5.7 lakhs of the asking price" — about 9% of the typical INR 62-lakh listing. Ridge achieves 5.74; a median-only baseline scores 17.7. The limit leaves ~20% headroom yet rejects clearly worse models.
Accuracy: RMSE	≤ 9.0 lakhs	Penalises occasional large misses that MAE averages away. Ridge achieves 7.52.
Safe promotion	≤ 2% worse than serving model	A candidate must beat the absolute limits and not regress against the incumbent on the same validation rows.
Final test	reported once	Reserved 800 listings, never used for selection: MAE 5.775, RMSE 7.506 lakhs.
Latency / errors	p95 ≤ 200 ms; errors ≤ 1%	Interactive single-listing estimates. Measured on the live free host (section 06).
Freshness	review if model > 30 days old	Model age is exported as a metric; drift or performance alerts trigger a retraining run through all gates.
Reproducibility	MAE within 1e-6	Seed 42, locked dependencies, hashed data and code snapshots.

02 / Architecture mapped to the MLOps lifecycle



Lifecycle stage	Components (where in the repository)
Problem framing, configuration	config.yaml validated on load by capstone/config.py (pydantic, extra keys forbidden); no hard-coded paths, thresholds or hyperparameters
Data management and versioning	Git-tracked CSVs; ingest snapshots and SHA-256 hashes every input; data hash tagged on each MLflow run
Model development	features/train/evaluate stages; experiments/ baseline comparison (Ridge vs gradient boosting vs random forest vs dummy)
Validation and governance	data gate (validate), model gate (evaluate), MLflow registry with lifecycle tags and champion alias
CI/CD and deployment	.github/workflows: ci.yml (PR), release.yml (main → image → release → Render), rollback.yml; Dockerfiles; render.yaml
Monitoring and feedback	monitor stage (3-batch drift + labelled error), /metrics for Prometheus, Grafana dashboard, live dashboard tabs

03 / Pipeline and gates — the workflow graph is the STAGES list in capstone/core.py

Eight stages run in order; each has declared inputs and outputs, persists its result and artifact hashes to the run directory and MLflow, and can be re-run alone (`python -m capstone stage <name> --run <dir>`). Re-running a stage clears its descendants and any changed upstream artifact is refused, so a stale gate result can never be reused. A deployed run is immutable.

Data gate (validate): exact schema, unique IDs, allowed categories, numeric ranges and integer types, ≥100 rows, ≤8% missing per column (observed 4%). **Model gate (evaluate):** MAE ≤ 7 and RMSE ≤ 9 lakhs, and ≤ 2% worse than the serving model. Deploy then re-checks model bytes, incumbent version and a finite prediction before atomically switching the serving pointer under a file lock. **Safe-failure evidence** (Real HTTP over TCP to a separate Uvicorn process at http://127.0.0.1:8765): bad data (20% of Age_Years blanked) stopped at validate; a median-only dummy model (MAE 17.73) stopped at evaluate; during both runs the endpoint answered 115 requests with 0 errors and kept model v1.

04 / Tracking, registry and lineage

Every run is an MLflow run carrying configuration, code snapshot hash, Git commit, data hash, stage records and metrics; failed runs are kept. Registration adds a model version tagged *validated*; only the deploy and rollback functions set *production*/*archived* tags and the *champion* alias (current MLflow replaces deprecated stages with aliases). **Live lineage:** the public `/lineage` endpoint and the dashboard's Lineage tab trace the serving model to its model hash, MLflow run, Git commit, training-data hash and full configuration.

05 / CI/CD, release and rollback

Pull requests run *Quality checks*: tests plus the pipeline to the model gate. PR #2 ($\alpha = 1000$) **failed** because the gate measured MAE 8.77 > 7; the follow-up commit restored $\alpha = 10$ and passed. Merging to main runs *Build and release*: tests, a Docker Compose smoke test, a linux/amd64 image pushed to Docker Hub tagged with the full commit SHA (and labelled with it), a GitHub release `build-<sha>`, then deployment of that exact commit to Render, which fails the job unless live /lineage reports the same commit. So any running container links back to a commit. **Rollback:** the *Roll back Render deployment* workflow redeploys any commit that already has a release; tested on 2026-10-23: live service rolled back from `6f42634` to `d1fa909` in 2 min and forward again in 2 min, each verified by live /lineage.

06 / Monitoring, alerts and response

Signal	Alert rule (justification)	Batch 1	Batch 2	Batch 3
Input drift	numeric KS D > 0.10, categorical total variation > 0.10 (effect size: ~10% of the distribution has moved)	none	Area	Area, Bedrooms, Bathrooms, City Zone
Missingness	change > 3 percentage points (3× the observed rate swing)	none	none	none
Performance	MAE > 7 lakhs or > 1.25 × validation MAE (labels arrive with each batch)	5.56 — OK	6.87 — OK	**13.11 — ALERT**
Service	p95 ≤ 200 ms, errors ≤ 1%, model age ≤ 30 days (Prometheus rules; dashboard Operations tab)	see load test below		

When an alert fires the serving model is never replaced automatically: the team investigates, then starts a new pipeline run with fresh labelled data; the candidate goes live only through all gates. Batch 3 shifts towards smaller, central properties and doubles the error, so it should trigger retraining. **Live operations (load test, 2026-10-22):** 300 predictions to Render, 0 errors; sequential p95 213 ms end to end, 857 ms with 5 concurrent clients — the error target is met, the 200 ms latency target is **not** met on the free single-worker host.

07 / Tool choices and limits

The brief allows justified free substitutes. A small persisted Python DAG replaces Prefect/ZenML (explicit dependencies, artifact hashes, reruns, MLflow history, no extra service). Git plus SHA-256 snapshots replace DVC for 0.3 MB of supplied CSVs. SciPy effect-size tests replace Evidently. Render Free replaces a paid cloud host: it sleeps when idle, keeps no runtime state, and each build retrains from scratch, so cross-release incumbent comparison is proven locally but not on Render. The public API has no authentication; registry alias and serving pointer are not one transaction. Fresh-clone reproduction: 16 passed, 3 warnings in 45.07s; validation MAE reproduced within 6e-15.